

AUTHORITATIVE SDLC & DEVOPS GOVERNANCE BLUEPRINT

Rankuno AI Engine - Master System & DevOps Infrastructure

Permanent Engineering Standards: Antigravity Stack, Mandatory 8-Step SDLC, GitHub & CI/CD Pipelines, Cloud Deployment, and Observability

1. Core Repository Principles & Zero-Legacy Mindset

Every module, connector, and tool created within Rankuno's AI infrastructure must adhere to three foundational principles:

- **Zero-Legacy Mindset:** Every system is clean, modular, and built from scratch with zero tech debt.
- **Strict Pydantic Data Contracts:** All tool interfaces, data models, and API responses MUST be strongly typed using Pydantic v2 `BaseModel`.
- **No Documentation Drift:** Any modification to tools, APIs, environment variables, or module structures MUST be reflected immediately in `README.md` and `docs/ARCHITECTURE.md`.

2. Google Antigravity Agentic Architecture Stack

The platform is organized into 7 distinct operational layers:

Layer Name	System Component	Google Antigravity Equivalent	Operational Function in Rankuno
Reasoning Model	LLM Brain	Gemini 3.6 Flash / Pro	Processes natural language instructions, analyzes edge cases, and synthesizes reports.
Agent Loop	Decision Orchestrator	Antigravity Core Loop	Runs autonomous <code>Plan → Act → Observe → Reason</code> execution loops.
Subagents	Specialized Workers	<code>invoke_subagent</code> API	Spawns isolated background agents for research, refactoring, and test suite verification.
Skills	Procedural Knowledge	<code>skills/<name>/SKILL.md</code>	Houses step-by-step operational workflows (e.g. <code>antigravity-sdlc-flow</code>).
Tools	Native Capabilities	System Tool Interfaces	Atomic execution hands: <code>read_file</code> , <code>write_file</code> , <code>run_command</code> , <code>grep_search</code> .
MCP Connectors	External APIs	Model Context Protocol	Connects platform to Google Search Console API, Google Ads API, Ahrefs, and SERP APIs.
Guardrails	HITL & Permissions	Permission Controls	Prevents unauthorized live site modifications or unapproved API financial spend.

3. Mandatory 8-Step SDLC Execution Protocol

Every feature, tool, or refactor MUST strictly follow this execution loop:

1. **Step 1: Investigation & Requirement Discovery** — Audit domain rules, inspect dependencies, and verify code logic.
2. **Step 2: System Architecture & Data Schema Blueprint** — Formulate Pydantic contracts and component boundaries.
3. **Step 3: Human-in-the-Loop (HITL) Review Checkpoint** — Present proposed architecture to operator for explicit approval.
4. **Step 4: Reasoning-Driven Implementation Plan** — Detail line-by-line file changes and logic.
5. **Step 5: Security, Rate-Limit & Financial Audit** — Audit for anti-bot blocks, rate limits, API spend, and breaking changes.
6. **Step 6: Step-by-Step Modular Implementation** — Write clean, strictly typed, modular code.
7. **Step 7: Automated Verification & Testing** — Execute unit tests (`pytest`); never declare success without passing tests.
8. **Step 8: README & Architecture Drift Audit** — Immediately update `README.md` and `docs/ARCHITECTURE.md` .

4. Full DevOps Infrastructure, Cloud & CI/CD Standards

To ensure high availability, security, and seamless deployment after code verification, Rankuno enforces an automated DevOps lifecycle:



4.1 GitHub Version Control Standards

- **Main Branch Protection:** Direct pushes to `main` are disabled. All changes must arrive via Pull Request.
- **Semantic Branch Naming:** `feat/module-name` , `fix/bug-name` , `docs/update-name` , `test/feature-name` .
- **Automated PR Status Checks:** Pull Requests cannot be merged unless all CI workflow jobs pass 100%.

4.2 CI/CD Pipeline Configuration (GitHub Actions)

Every PR triggers an automated GitHub Actions pipeline performing 4 verification steps:

1. **Linting:** Enforces PEP8 code style via `ruff check` .
2. **Type Checking:** Enforces strict typing via `mypy src/`
3. **Unit Testing:** Executes full test suite via `pytest --cov=src tests/`
4. **Security Scan:** Scans commits for exposed API keys via `trufflehog`

4.3 Cloud Server Architecture & Deployment

- **Containerization:** Every microservice is packaged into lightweight Docker containers using multi-stage builds.
- **Cloud Infrastructure:** Serverless API endpoints deployed to AWS Cloud Run / Railway / Vercel. Async scraping background tasks managed via Redis queues.
- **Observability & Alerts:** Structured JSON logs sent to cloud logging providers with automated Sentry exception tracking.